

AI Concierge / Intent Engine Backend Architecture & Development Playbook

Hackathon working document for internal implementation

Purpose: capture the complete backend-improvement discussion and turn it into an actionable, step-by-step engineering plan for use with your organization's internal ChatGPT.

1. Executive Summary

The prototype should evolve from a basic LLM call with large prompt context into an enterprise-style, tool-orchestrated AI system. The core idea is to minimize user effort, dynamically understand intent, call only the tools needed for the request, retrieve only relevant card/benefit/event context, optimize token usage, and generate a grounded personalized response.

The target experience is an intelligent Amex-oriented concierge / intent engine: a customer can either type naturally or click suggested prompt chips such as "Plan my evening", "Show exclusive dining experiences", or "Find events for my travel dates". The backend then interprets intent, invokes the right tools, retrieves and filters enterprise data, builds a compact context package, calls the LLM, and returns a personalized answer with clear evidence.

2. What the Manager Is Asking to Improve

Move beyond simple prompt stuffing: Do not pass every card, benefit, service, and merchant record into a single prompt. Retrieve and send only what is needed.

Optimize context window and token consumption: Reduce repeated or irrelevant context so cost, latency, and scale remain manageable.

Use advanced prompting techniques: Introduce step-back prompting, structured prompt templates, query rewriting, and bounded reasoning.

Use hybrid retrieval: Combine semantic retrieval with keyword/metadata filters and re-ranking.

Introduce specialized tools: Use separate tools/services for web discovery, events, places, coordinates, routing/travel estimates, Amex benefits, offers, merchant data, and experiences.

Improve customer UX: Add 5-6 suggested prompt chips below the input so users do not need to know what to ask.

Support exclusive Amex events/experiences: Integrate an approved internal source when available; otherwise use clearly marked synthetic data for the hackathon.

Highlight efficiency: Show token optimization, caching, model routing, and cost-aware architecture as a deliberate product benefit.

3. Product Experience

The goal is minimal customer input. The user should be able to discover value even when they do not know what benefits or capabilities exist.

Suggested Prompt Chips

Plan my evening

Show exclusive Amex experiences

Find dining benefits near me

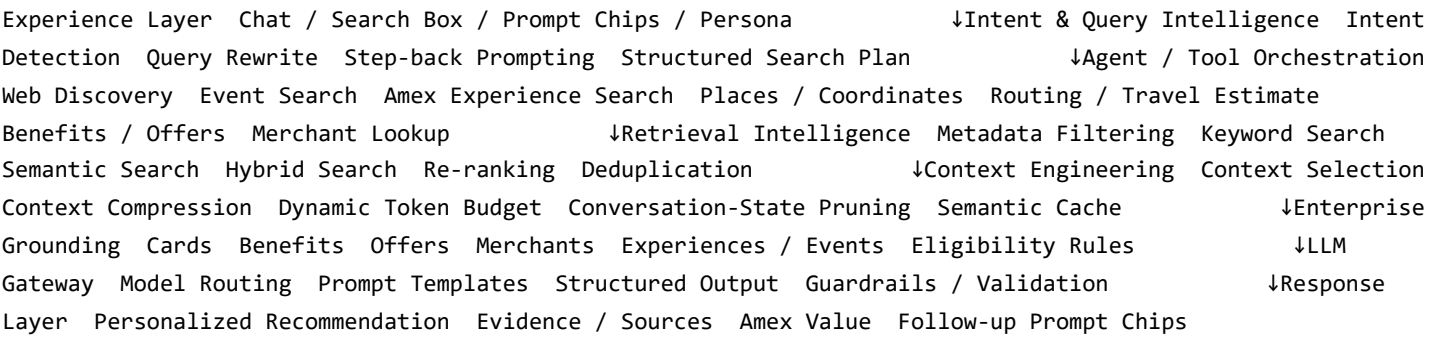
What can I do during my trip?

Where can my card give me extra value?

Show events for my travel dates

Prompt chips should ideally be context-aware. They may vary by persona, selected card, location, travel dates, prior interaction state, or eligible benefits. They should not be static hard-coded labels if the backend already has enough information to personalize them.

4. Target Backend Architecture



5. Why This Is Better Than the Current Basic Pattern

Area	Basic Prototype	Improved Architecture
Context	All data stuffed into prompt	Retrieve only relevant records
Search	One generic query	Tool-based and intent-driven
Retrieval	Semantic only or direct lookup	Hybrid retrieval + filters + reranking
Prompting	Single prompt	Structured templates + step-back/query rewrite
Cost	Repeated full LLM calls	Caching + smaller contexts + model routing
Accuracy	LLM sees noisy context	Grounded, filtered, validated context
User UX	User must know what to ask	Dynamic prompt chips
Scalability	Tight coupling	Provider/tool abstractions

6. Detailed Request Flow

1. User types or clicks a prompt chip
2. Backend detects intent and entities
3. Optional step-back/query rewrite creates a broader, cleaner task representation
4. Router creates a structured search/tool plan
5. Relevant tools run in parallel where possible
6. Candidate data is normalized
7. Metadata filters remove obviously irrelevant records
8. Hybrid retrieval identifies semantically relevant records
9. Re-ranker keeps only top candidates
10. Context builder selects the minimum evidence needed
11. Token budget manager compresses/prunes context

12. Semantic cache is checked before an expensive LLM call
13. Model router selects the appropriate model
14. LLM generates a grounded response using structured context
15. Output validator checks schema and required evidence
16. UI renders recommendations, value, and next prompt chips

7. Context Engineering Strategy

This is the most important backend improvement. The system should not send the full card-benefit-service universe to the LLM. The backend should construct a compact, purpose-built context package for every request.

7.1 Recommended Context Pipeline

User Query ↓Intent / Persona / Location / Date Extraction ↓Step-back or Query Rewrite (only when useful)
↓Metadata Filters ↓Hybrid Retrieval ↓Top-K Candidate Set ↓Re-ranking ↓Context Compression ↓Token
Budget Check ↓LLM

7.2 Example

User query: "What premium dining benefits can I use in Rome tonight?"

Extract metadata: city=Rome, category=Dining, time=Tonight, card/profile=current user.

Apply metadata filters against eligible benefit/merchant records.

Run semantic retrieval for premium dining / restaurant experience intent.

Retrieve perhaps 20 candidates.

Re-rank to the best 3-5 records.

Pass only those records plus essential user/card context to the LLM.

8. Step-Back Prompting

Step-back prompting can be used as a query-enhancement technique. The idea is to first convert a narrow user question into a more general decision question, then use that improved representation to guide retrieval and reasoning.

Example user query: "Where can I go tonight and get the best value from my card?"

Possible step-back representation: "Identify suitable evening activities based on location, time, card eligibility, benefit relevance, merchant match, and user preferences."

Use step-back prompting selectively. It should improve retrieval or reasoning, not become an extra LLM call for every request. For simple deterministic queries, skip it.

9. Hybrid Retrieval Design

Hybrid retrieval should combine deterministic filtering with semantic similarity. This is important because benefits and offers often have strict eligibility constraints that semantic search alone may not respect.

Step 1 - Hard / metadata filters card product persona city / country date validity category merchant
type eligibilityStep 2 - Keyword / lexical retrieval exact offer names merchant names program termsStep 3

- Semantic retrieval intent similarity natural-language benefit matching
Step 4 - Re-ranking business relevance eligibility semantic score freshness confidence
Step 5 - Top-K context only

10. Tool / Service Catalog

Component	Responsibility
IntentService	Detect intent, entities, persona and task type
PromptChipService	Generate context-aware suggested prompts
WebDiscoveryTool	Discover public events or information where policy permits
EventProvider	Search events / experiences
AmexExperienceProvider	Approved internal experiences/events source, or synthetic hackathon adapter
PlacesProvider	Resolve places, coordinates, address and venue identity
RoutingProvider	Travel distance/time estimates
CardProvider	Card profile and card metadata
BenefitProvider	Card benefit definitions and eligibility
OfferProvider	Offer catalogue / eligibility
MerchantProvider	Merchant identity and relationships
HybridRetriever	Filter + keyword + semantic retrieval
Reranker	Reduce retrieved set to best evidence
ContextBuilder	Build compact LLM input context
SemanticCache	Reuse similar prior computations

ModelRouter	Choose small/large model based on complexity
LLMGateway	Centralize model invocation, schemas, retries, logging
ResponseValidator	Validate structure, required grounding, safety constraints

11. Exclusive Amex Events / Experiences

The conversation specifically raised the need to surface events that may be exclusive to Amex card members. Treat this as a first-class provider, not as free-form LLM knowledge.

Preferred implementation: integrate an approved internal event/experience catalogue or API if one exists and access is authorized. For the hackathon, if such access is unavailable, build an AmexExperienceProvider interface backed by a clearly labeled synthetic dataset. The rest of the system should not care whether the provider is internal or synthetic.

Suggested event fields: event_id, title, description, city, venue, date/time, eligible cards, booking channel, inventory/availability if known, benefit/value text, terms reference, and source timestamp.

12. Token, Latency and Cost Optimization

Semantic cache: Reuse results for sufficiently similar requests when underlying data is still fresh.

Retrieval before generation: Never send the whole knowledge base to the model.

Top-K control: Limit final context to the few records required to answer.

Re-ranking: Remove noisy results before prompt construction.

Context compression: Summarize or extract only required fields from large documents.

Conversation pruning: Do not resend irrelevant chat history.

Dynamic token budgets: Set context and output budgets based on task complexity.

Model routing: Use smaller/cheaper models for classification, extraction and simple answers; stronger models for complex planning.

Parallel tools: Run independent search/place/event calls concurrently to reduce latency.

Deterministic code: Use code for filtering, calculations, eligibility checks and matching rather than spending LLM tokens.

13. Model Routing Strategy

Do not use the most expensive model for every stage. A practical router can classify requests into lightweight, standard, or reasoning-heavy paths.

Task	Recommended Approach	Why
Intent classification	Small/fast model or deterministic classifier	Low complexity
Entity extraction	Small structured-output model	Predictable schema
Query rewrite	Small/standard model	Short reasoning step
Benefit retrieval	Deterministic retrieval	Do not use generation
Eligibility	Rules/service	Needs determinism
Complex itinerary planning	Stronger reasoning model	Multi-step trade-offs
Final explanation	Standard model	Grounded generation

14. Prompt Engineering Strategy

Use structured prompt templates instead of ad-hoc concatenation.

SYSTEM- Role and boundaries- Grounding rules- Do not invent benefits/offers/events- Use only supplied evidence for enterprise facts
TASK- The specific user goal
USER CONTEXT- Persona- Card- Location- Travel dates- Preferences
RETRIEVED CONTEXT- Only top-ranked relevant benefits/offers/events/merchants
TOOL RESULTS- Places / route / event / availability outputs
OUTPUT CONTRACT- JSON or strongly structured response- Recommendation- Reason- Value- Evidence IDs- Suggested next prompt chips

15. Grounding and Hallucination Controls

LLM must not invent offers, eligibility, merchant matches, event exclusivity, or card entitlements.

Enterprise facts must reference retrieved records or approved service outputs.

Eligibility should be determined by rule/service logic, not natural-language inference alone.

Use structured output schemas so missing evidence can be detected.

If evidence is insufficient, return an explicit uncertainty/fallback state.

Keep audit metadata for tool calls, retrieved record IDs, and model responses.

16. Suggested Python Backend Structure

```

backend/  app/      api/      chat.py      suggestions.py  health.py  models/      requests.py
responses.py  search_plan.py  benefit.py    offer.py      event.py      services/    intent_service.py

```

query_enhancer.py prompt_chip_service.py context_builder.py ranking_service.py
response_validator.py retrieval/ hybrid_retriever.py metadata_filter.py semantic_search.py
reranker.py tools/ web_discovery.py events.py amex_experiences.py places.py routing.py
benefits.py offers.py merchants.py llm/ gateway.py model_router.py prompts/
schemas/ cache/ semantic_cache.py orchestration/ planner.py workflow.py state.py
observability/ logging.py metrics.py tracing.py tests/ unit/ integration/

17. Step-by-Step Development Roadmap

Phase 0 - Baseline the current prototype

Document current request/response flow.

Identify where the LLM is called.

List all context currently appended to the prompt.

Measure current tokens, latency, and number of external calls.

Preserve a baseline demo before refactoring.

Phase 1 - Introduce typed service boundaries

Create provider/service interfaces for cards, benefits, offers, merchants, events, places and routing.

Move hard-coded/mock data behind providers.

Keep behavior unchanged while creating clean seams.

Phase 2 - Build intent detection and SearchPlan

Create structured intent schema.

Extract location, dates, categories, persona and selected card.

Return JSON, not free-form reasoning.

Add unit tests for common hackathon queries.

Phase 3 - Add dynamic prompt chips

Create PromptChipService.

Use persona + selected card + location/session context.

Return 5-6 suggestions in API payload.

Render chips below search/chat input.

Phase 4 - Implement hybrid retrieval

Add metadata filters.

Add lexical/keyword retrieval.

Add vector/semantic retrieval if infrastructure exists.

Merge scores and deduplicate.

Introduce top-K limits.

Phase 5 - Add re-ranking

Re-rank retrieved benefit/event/merchant candidates.

Prefer deterministic/business criteria first.

Use a model reranker only if needed.

Measure retrieval precision.

Phase 6 - Add step-back/query enhancement

Invoke only for ambiguous or multi-part requests.

Convert query into retrieval-oriented representation.

Compare retrieval quality with and without the technique.

Avoid unnecessary additional model calls.

Phase 7 - Build ContextBuilder

Construct a minimal context package.

Include only top-ranked records and required fields.

Prune duplicate text.

Add token estimation before model invocation.

Phase 8 - Add semantic caching

Define cacheable request types.

Include user/card eligibility and data freshness in cache key.

Add hit/miss metrics.

Never reuse stale dynamic event/offer information blindly.

Phase 9 - Add model routing

Use cheaper path for classification/extraction.

Use stronger model only for complex planning.

Create config-based model mappings.

Track tokens and cost by stage.

Phase 10 - Add tools for events/places/routing

Integrate approved providers available in the organization.

Use synthetic adapters where real systems are unavailable for the hackathon.

Normalize all tool outputs.

Add timeout, retry and partial-failure behavior.

Phase 11 - Add Amex experience provider

Create interface for exclusive events/experiences.

Use approved internal source when accessible.

Otherwise use synthetic curated demo records.

Filter by card eligibility/date/location before generation.

Phase 12 - Add grounded response generation

Create strict prompt template.

Require evidence IDs.

Validate output schema.

Reject/retry responses that cite unavailable facts.

Phase 13 - Observability and cost dashboard

Track per-request latency.

Track tokens by stage.

Track cache hits.

Track retrieval candidate counts.

Track model route selected.

Capture tool errors without sensitive data.

Phase 14 - Demo polish

Stream pipeline progress to UI.

Show why recommendation was selected.

Display benefit/value summary.

Add follow-up prompt chips.

Prepare before/after token/cost comparison for judges.

18. Metrics to Track

Average prompt input tokens per request

Average output tokens per request

Token reduction versus baseline prompt-stuffing approach

LLM calls per request

Cache hit rate

Retrieval precision / relevance of top-K

Reranker reduction ratio

P50 / P95 end-to-end latency

Tool-call latency

Model cost per request where measurable internally

Percentage of answers with complete grounding/evidence

Fallback/error rate

Prompt-chip click-through rate during demo/usability testing

19. Demo Story

A strong hackathon demonstration should show the architectural intelligence, not only the final chat response.

User logs in under a persona and selected card.

UI presents personalized prompt chips.

User clicks: "Plan my evening in Rome".

Backend interprets location/date/activity intent.

Tool router searches events, experiences, places and travel options.

Benefit retrieval filters to eligible card benefits/offers only.

Hybrid search and re-ranking reduce the context to the most relevant records.

Context builder shows that only a small subset is sent to the LLM.

LLM generates a grounded plan with relevant card value.

UI returns follow-up chips such as "Make it cheaper", "Only Amex experiences", or "Add dinner".

Developer/debug panel optionally shows tokens saved, cache status, tools used and latency.

20. What to Tell Stakeholders

Recommended narrative: "We are not simply wrapping an LLM API. The system uses an intent-driven orchestration layer to call specialized tools, hybrid retrieval and re-ranking to select only the most relevant enterprise context, and context optimization to reduce token consumption and latency. The LLM is used for language understanding and personalized reasoning, while eligibility, offers, merchant data and experience facts remain grounded in approved services."

21. Starter Prompt for Your Internal ChatGPT

Act as a Principal AI Engineer helping me incrementally upgrade an existing hackathon prototype. Context:- Frontend is React/Node-based.- Backend is Python.- The current solution already makes LLM calls but sends too much context directly in the prompt.- We need to move toward an enterprise AI architecture with intent routing, specialized tools, hybrid retrieval, re-ranking, context compression, semantic caching, model routing, structured prompts, grounding and observability.- We also want dynamic prompt chips so users can discover capabilities with minimal typing.- Domain tools may include card data, benefits, offers, merchants, events/experiences, web discovery, places/coordinates, and travel estimation.- For Amex-exclusive events or proprietary data, use only approved

internal sources available to this environment. If unavailable, create clearly labeled synthetic adapters for the hackathon; do not invent real entitlements. Engineering principles: 1. Do not rewrite the product from scratch. 2. Inspect the repository first. 3. Preserve current working behavior. 4. Introduce clean interfaces incrementally. 5. Keep deterministic business rules outside the LLM. 6. Retrieve only the context required for the request. 7. Prefer metadata filtering + lexical/semantic hybrid retrieval + re-ranking. 8. Use step-back/query rewriting only when it adds value. 9. Track tokens, latency, cache hits and model selection. 10. Use structured JSON schemas for LLM outputs. 11. Add tests after every feature. First task: Inspect the existing codebase and tell me:- current architecture- where LLM calls happen- how context is currently built- where static/mock data lives- current API endpoints- current card/benefit/merchant/event models- exact files that should change first. Then propose only Phase 1 changes: introduce provider/service boundaries without changing user-visible behavior. Do not jump ahead until Phase 1 is complete and tested.

22. Internal ChatGPT Step-by-Step Working Pattern

For implementation, do not paste the entire roadmap and ask the coding assistant to build everything at once. Use one phase per conversation/task and require it to inspect, implement, test, and summarize before moving to the next phase.

Paste the starter prompt once.

Ask it to inspect the repository and return architecture + gap analysis.

Approve/adjust Phase 1 only.

Ask it to implement Phase 1 and run tests.

Request a concise change log and exact files modified.

Proceed to Phase 2 using the same architecture document as reference.

At each stage, ask for token/latency impact when relevant.

Before demo, run an end-to-end test across at least 5 representative user journeys.

23. Guardrails for Organization Use

Do not paste confidential production credentials, secrets, private keys, or customer data into any assistant prompt.

Use organization-approved internal models, APIs, connectors and data stores.

Do not fabricate proprietary card benefits, offers, eligibility, or exclusive events.

Treat synthetic hackathon data as synthetic and label it accordingly.

Keep audit logs free of sensitive customer/payment data.

Follow internal security, legal, privacy, model-risk and architecture-review requirements before moving beyond prototype use.

24. Definition of Done for the Improved Prototype

User can interact through natural language or dynamic prompt chips.

Backend creates a structured intent/search plan.

Specialized tools are called based on intent.

Hybrid retrieval reduces enterprise context before generation.

Re-ranking selects only the most useful evidence.

Context builder enforces a token budget.

Step-back/query enhancement is available for complex requests.

Semantic cache and model routing are implemented or demonstrably stubbed.

Responses are grounded in tool/provider data.

Exclusive Amex experience capability has a provider abstraction.

Observability captures tokens, latency, cache status and selected tools.

UI can show recommendation, Amex value, and suggested next actions.

A before/after comparison demonstrates that the new architecture is more efficient than the original prompt-stuffing implementation.